

Compiladores en el mundo real: de SiteLang a un deploy en Vercel

Construcción de Compiladores — Universidad del Valle de Guatemala
Laboratorio 3 — Opción D (GitHub + Vercel)
Esteban Cárcamo (23016) y Nicolás Concuá (23197) — 31 de julio de 2026

1. Introducción

Herramientas como Terraform, Vercel CLI o Netlify CLI suelen percibirse como utilidades de operaciones y no como compiladores; sin embargo, su estructura interna es la de un compilador clásico: leen un archivo de configuración escrito en un lenguaje de dominio específico (DSL), lo tokenizan, lo parsean, construyen una representación intermedia del estado deseado y ejecutan un backend que la traduce en llamadas a APIs REST. Este trabajo replica ese pipeline a escala reducida: se diseñó un DSL propio, *SiteLang*, se generó su lexer y parser con ANTLR, y se implementó un listener que produce HTML y publica un sitio real en internet usando las APIs de GitHub y Vercel. La distancia entre una gramática de curso y una herramienta de despliegue en uso real es, esencialmente, el backend.

2. Pipeline del compilador

El programa de entrada es `site.sl`: un archivo declarativo con el nombre del sitio, atributos globales (`title`, `description`, `theme`) y bloques `page` con sus propios atributos (`hero`, `about`, `contact`). La gramática `SiteLang.g4` define las reglas léxicas (`STRING`, `NUMBER`, `BOOLEAN`, `IDENTIFIER`, más `COMMENT` y `WS` descartados con `-> skip`) y las sintácticas (`site`, `siteBody`, `siteAttr`, `page`, `pageBody`, `pageAttr`, `expr`).

```
site.sl
|--> [ Lexer: SiteLangLexer ]      tokens: 'site', STRING, '{', IDENTIFIER, '=' ...
|--> [ Parser: SiteLangParser ]    reglas: site -> siteBody -> (siteAttr | page)*
|--> [ Parse tree ]
|--> [ ParseTreeWalker + SiteDeployListener ] estado: site_name, site_attrs{}, pages{}
|--> [ Generacion de codigo: index.html ]
      |--> POST api.github.com/user/repos
      |--> PUT  api.github.com/repos/{full_name}/contents/index.html
      |--> POST api.vercel.com/v13/deployments (target: production)
          |
          v
      URL publica en produccion
```

Figura 1. Pipeline de SiteLang, del archivo fuente a la URL desplegada.

`Driver.py` orquesta las fases: valida que `GITHUB_TOKEN` y `VERCEL_TOKEN` existan en el entorno (cargados desde un `.env` excluido del control de versiones), construye un `FileStream` sobre el fuente, lo pasa al lexer, alimenta el `CommonTokenStream` al parser, invoca la regla inicial `site()` y recorre el parse tree resultante con un `ParseTreeWalker` y una instancia de `SiteDeployListener`.

El listener concentra el análisis semántico y la generación de código: `enterSite` captura el nombre del sitio, `enterSiteAttr` acumula los atributos globales, y `enterPage/enterPageAttr/exitPage` mantienen la página actual para asociar cada atributo con su bloque —un análogo simplificado de una tabla de símbolos con ámbitos anidados—. Al finalizar, `deploy()` emite el artefacto: `_generate_html()` construye un `index.html` completo, con la paleta derivada del atributo `theme` (`dark` o `light`) y secciones que solo se emiten si el atributo respectivo fue declarado.

3. Comparación con `vercel deploy`

Al ejecutar `vercel deploy`, la CLI hace cuatro cosas: lee la configuración del proyecto (`vercel.json` y metadatos locales, que son también un DSL declarativo); la resuelve contra un

modelo interno del despliegue deseado; construye los artefactos, ejecutando el build del framework o recolectando los archivos estáticos; y, autenticada con un token, los envía a la API REST de Vercel, que devuelve una URL de producción. La CLI no despliega por sí misma: traduce una descripción declarativa en llamadas HTTP. Nuestro compilador ejecuta esa misma secuencia con piezas propias:

Fase	Vercel CLI	Este compilador
Lectura de la configuración	Parseo de <code>vercel.json</code> y del proyecto	Lexer y parser generados por ANTLR sobre <code>site.sl</code>
Modelo interno	Objeto de despliegue resuelto	Parse tree y estado del listener (<code>site_attrs</code> , <code>pages</code>)
Artefactos	Build del framework / archivos estáticos	<code>_generate_html()</code> emite <code>index.html</code>
Publicación	POST a la API de deployments con token	POST <code>/v13/deployments</code> con <code>VERCEL_TOKEN</code>

La coincidencia decisiva es que ambos terminan en el mismo endpoint: `_deploy_to_vercel()` envía a `api.vercel.com/v13/deployments` un JSON con el nombre del proyecto, el arreglo `files` con el contenido de `index.html`, `projectSettings` y `target: "production"` —la misma interfaz que consume la CLI oficial—. Se añade un paso que Vercel CLI delega a git: `_create_github_repo()` y `_push_to_github()` crean el repositorio y suben el archivo mediante la API REST de GitHub (POST `/user/repos` y PUT `/repos/.../contents/index.html`, con el contenido en base64), sin ejecutar un solo comando de git.

Las simplificaciones son igualmente explícitas. SiteLang es un lenguaje mucho menor: sus valores se limitan a cadenas, números y booleanos, sin expresiones compuestas, variables, referencias entre bloques ni módulos. No hay una CLI con subcomandos ni gestión de dominios, entornos de preview o rollbacks: toda ejecución despliega a producción. Tampoco existe una fase de plan y diferencia contra el estado remoto —el equivalente de `terraform plan`—, por lo que cada corrida es una creación nueva, y el manejo de errores se limita a `raise_for_status()` de `requests`, sin reintentos ni transaccionalidad. Finalmente, aunque la gramática admite varias páginas, solo `index` se materializa en HTML: el frontend del compilador es hoy más expresivo que su backend.

4. El rol de ANTLR

ANTLR funciona aquí como generador de compiladores. De la única especificación `SiteLang.g4` produce `SiteLangLexer.py`, `SiteLangParser.py` y la clase base `SiteLangListener.py`: todo el frontend —análisis léxico, análisis sintáctico y construcción del parse tree— junto con los métodos `enterX/exitX` que definen el contrato de recorrido. Lo que no genera, y es el aporte propio de este trabajo, es la semántica: `SiteDeployListener` hereda de esa clase base y actúa como backend, interpretando los nodos, generando código y produciendo efectos sobre el mundo real. Esta separación permite modificar el lenguaje sin reescribir el traductor, y viceversa.

5. Conclusión

Al conectar un listener con dos APIs REST, un árbol sintáctico deja de ser una estructura de datos académica y se convierte en un sitio accesible públicamente. La diferencia respecto de `vercel deploy` o `terraform apply` es de escala y madurez —cobertura del lenguaje, manejo de estado, robustez ante fallos—, no de naturaleza: los tres leen un DSL, lo parsean, construyen una representación intermedia del estado deseado y ejecutan un backend que emite llamadas a una API. Entender ese patrón permite leer esas herramientas no como cajas negras, sino como compiladores cuyo código objeto es una petición HTTP.